

Preparation Notebook

Setup Environment

```
In [71]: # DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipynrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

Student Information

```
In [72]: group_name = ""
student_name = "Yi An Tsai"
student_id = "25532767"
```

```
In [73]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='group_name', value=group_name)
```

group_name

```
In [74]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Yi An Tsai

```
In [75]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

25532767

0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

```
In [76]: # <Student to fill this section and then remove this comment>
```

0.b Import Packages

```
In [78]: # DO NOT MODIFY THE CODE IN THIS CELL
import pandas as pd
import altair as alt
```

```
In [79]: pd.set_option('display.max_columns', None)
```

A. Feature Selection

A.0 Load Data

```
In [80]: # DO NOT MODIFY THE CODE IN THIS CELL
# Load datasets
try:
    customer_df = pd.read_csv(at.folder_path / "customer.csv")
```

```

person_df = pd.read_csv(at.folder_path / "person.csv")
product_category_df = pd.read_csv(at.folder_path / "product_category.csv")
product_cost_history_df = pd.read_csv(at.folder_path / "product_cost_history.csv")
product_list_price_history_df = pd.read_csv(at.folder_path / "product_list_price_history.csv")
product_sub_category_df = pd.read_csv(at.folder_path / "product_sub_category.csv")
product_df = pd.read_csv(at.folder_path / "product.csv")
sales_order_detail_df = pd.read_csv(at.folder_path / "sales_order_detail.csv")
sales_order_header_df = pd.read_csv(at.folder_path / "sales_order_header.csv")
sales_territory_df = pd.read_csv(at.folder_path / "sales_territory.csv")
special_offer_product_df = pd.read_csv(at.folder_path / "special_offer_product.csv")
special_offer_df = pd.read_csv(at.folder_path / "special_offer.csv")
store_df = pd.read_csv(at.folder_path / "store.csv")
unit_measure_df = pd.read_csv(at.folder_path / "unit_measure.csv")
except Exception as e:
    print(e)

```

A.1 Approach 1 "Merge 7 tables"

Combine sales_order_detail_df and product_df as combined_df

```

In [81]: # check duplicate rows in advance
print(product_df.duplicated().sum())
print(customer_df.duplicated().sum())

382
3944

In [82]: # drop duplicate
product_df = product_df.drop_duplicates(keep='first')
customer_df = customer_df.drop_duplicates(keep='first')

In [83]: # merge sales_territory_df and customer_df on territory, add the location name and group
select_sales_territory_df = sales_territory_df[['territory_id', 'name', 'group']]
new_customer_df = customer_df.merge(select_sales_territory_df, on='territory_id', how='left' )

# rename avoid confusion
sales_territory_df = sales_territory_df.rename(columns={
    'name': 'sales_country',
    'group': 'sales_group'})

new_customer_df = new_customer_df.rename(columns={
    'name': 'customer_country',
    'group': 'customer_group'})

product_category_df = product_category_df.rename(columns={'name': 'category_name'})

# select required features
selected_product_df = product_df[['product_id', 'product_subcategory_id']]
selected_product_sub_category_df = product_sub_category_df[['product_subcategory_id', 'product_category_id']]
selected_sales_order_header_df = sales_order_header_df[[
    'sales_order_id',
    'online_order_flag',
    'customer_id',
    'territory_id',
    'order_date',
    'sub_total',
    'tax_amount',
    'freight',
    'total_due'
]]
selected_sales_territory_df = sales_territory_df[['territory_id', 'sales_country', 'sales_group']]
selected_new_customer_df = new_customer_df[['customer_id', 'customer_country', 'customer_group']]

# merge with selected features
combined_df = sales_order_detail_df.merge(selected_product_df, on='product_id', how='left') \
    .merge(selected_product_sub_category_df, on='product_subcategory_id', how='left') \
    .merge(product_category_df, on='product_category_id', how='left') \
    .merge(selected_sales_order_header_df, on='sales_order_id', how='left') \
    .merge(selected_sales_territory_df, on='territory_id', how='left') \
    .merge(selected_new_customer_df, on='customer_id', how='left')

# drop unused columns
combined_df = combined_df.drop(columns=['product_subcategory_id', 'product_category_id', 'product_id', 'territory_id'])

In [100]: # check if rows explode after merging
print(f'original shape of sales_order_detail_df: {sales_order_detail_df.shape}')
print(f'shape of combined_df: {combined_df.shape}', '\n')

if sales_order_detail_df.shape[0] == combined_df.shape[0]:
    print('No explode rows')
else:
    print('Explode rows')

original shape of sales_order_detail_df: (42100, 8)
shape of combined_df: (42100, 19)

No explode rows

In [85]: feature_selection_1_insights = """
Based on the business values, the features related to order history, item category, customer informations are required,
and merge them into one table for the next step.

```

```
The sales_order_detail_df is the base dataset, and check it rows after merging.
```

```
In [86]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_1_insights', value=feature_selection_1_insights)
```

```
feature_selection_1_insights
```

Based on the business values, the features related of order history, item category, customer informations are required, and merge them into one table for the next step.

A.z Final Selection of Features

```
In [87]: features_list = ['sales_order_id', 'sales_order_detail_id', 'order_quantity',
                        'product_id', 'special_offer_id', 'unit_price', 'unit_price_discount',
                        'line_total', 'category_name', 'online_order_flag', 'customer_id',
                        'sub_total', 'tax_amount', 'freight', 'total_due', 'sales_country',
                        'sales_group', 'customer_country', 'customer_group']
```

```
In [88]: feature_selection_explanations = """
Feature selection is implemented during the table merging process.
Columns related to product attributes are excluded to prevent data leakage, such as weight, size, color, and so on.
Other irrelevant columns are also dropped, such as due_date, and ship_date.

category_name (derived from product_category_df) is selected and renamed as the target variable.
sales_order_id, sales_order_detail_id, and customer_id are retained to identify and group orders for aggregation purposes.
sales_country (derived from sales_territory_df) and customer_country (also derived from sales_territory_df)
are included as location-based features for the model.

"""
```

```
In [89]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_explanations', value=feature_selection_explanations)
```

```
feature_selection_explanations
```

Feature selection is implemented during the table merging process. Columns related to product attributes are excluded to prevent data leakage, such as weight, size, color, and so on. Other irrelevant columns are also dropped, such as order_date, due_date, and ship_date. category_name (derived from product_category_df) is selected and renamed as the target variable. sales_order_id, sales_order_detail_id, and customer_id are retained to identify and group orders for aggregation purposes. sales_country (derived from sales_territory_df) and customer_country (also derived from sales_territory_df) are included as location-based features for the model.

B. Data Cleaning

B.1 Fixing "Order_date"

```
In [90]: # check Nan and type of "order_date"
combined_df['order_date'].info()          # no Nan, and object type
```

```
<class 'pandas.core.series.Series'>
RangeIndex: 42100 entries, 0 to 42099
Series name: order_date
Non-Null Count  Dtype
-----
42100 non-null  object
dtypes: object(1)
memory usage: 329.0+ KB
```

```
In [69]: # convert it into datetime type
combined_df['order_date'] = pd.to_datetime(combined_df['order_date'])

# verify success
combined_df['order_date'].info()
```

```
<class 'pandas.core.series.Series'>
RangeIndex: 42100 entries, 0 to 42099
Series name: order_date
Non-Null Count  Dtype
-----
42100 non-null  datetime64[ns]
dtypes: datetime64[ns](1)
memory usage: 329.0 KB
```

```
In [101]: data_cleaning_1_explanations = """
This case is time-sequence so it is converted into datetime type for the further addressing.
"""
```

```
In [102]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_1_explanations', value=data_cleaning_1_explanations)
```

```
data_cleaning_1_explanations
```

This case is time-sequence so it is converted into datetime type for the further addressing.

C. Split Datasets

```
In [27]: # distribution of "order_date"
combined_df['order_date'].describe()
```

```
Out[27]:
```

	order_date
count	42100
mean	2013-10-11 22:24:57.890736384
min	2011-05-30 22:00:00
25%	2013-07-30 22:00:00
50%	2013-11-16 23:00:00
75%	2014-03-03 23:00:00
max	2014-06-29 22:00:00

dtype: object

```
In [28]: # get the date cutoffs based on quantiles
train_cutoff = combined_df['order_date'].quantile(0.6) # 60% for train set
val_cutoff = combined_df['order_date'].quantile(0.8) # 20% (80%-60%) for val set, remaining 20% for test set.

# show the exact points
print(f'Train cutoff: {train_cutoff}')
print(f'Validation cutoff: {val_cutoff}')
```

Train cutoff: 2013-12-30 23:00:00
Validation cutoff: 2014-03-27 23:00:00

```
In [29]: # split based on "order_date"
training_df = combined_df[combined_df['order_date'] < train_cutoff] # datetime < 60% standard -> training_df
validation_df = combined_df[(combined_df['order_date'] >= train_cutoff) & (combined_df['order_date'] < val_cutoff)] # datetime = 60%~80% -> validation_df
testing_df = combined_df[combined_df['order_date'] >= val_cutoff] # datetime > 80% -> testing_df
```

```
In [92]: # the percentage of each set
print(f'Train size: {len(training_df)} ({len(training_df)/len(combined_df)*100:.1f}%)')
print(f'Val size: {len(validation_df)} ({len(validation_df)/len(combined_df)*100:.1f}%)')
print(f'Test size: {len(testing_df)} ({len(testing_df)/len(combined_df)*100:.1f}%)')
```

Train size: 24750 (58.8%)
Val size: 8837 (21.0%)
Test size: 8513 (20.2%)

```
In [103... data_splitting_explanations = """
Since products have their own selling periods, order_date is used as the splitting criterion to reflect actual transaction time,
rather than shipping or arrival time.

Moreover, 60% for training, 20% for validation,
and 20% for testing is used to ensure sufficient training data while maintaining a meaningful evaluation set.
"""
```

```
In [104... # Do not modify this code
print_tile(size="h3", key='data_splitting_explanations', value=data_splitting_explanations)
```

data_splitting_explanations

Since products have their own selling periods, order_date is used as the splitting criterion to reflect actual transaction time, rather than shipping or arrival time. Moreover, 60% for training, 20% for validation, and 20% for testing is used to ensure sufficient training data while maintaining a meaningful evaluation set.

D. Feature Engineering

```
In [33]: # DO NOT MODIFY THE CODE IN THIS CELL
# Create copy of datasets

try:
    training_df_eng = training_df.copy()
    validation_df_eng = validation_df.copy()
    testing_df_eng = testing_df.copy()
except Exception as e:
    print(e)
```

D.1 New Feature "No Feature Engineering at This Stage"

```
In [34]: # no feature engineering at this stage
```

```
In [107... feature_engineering_1_explanations = """
To show how the effect of feature engineering on baseline model , no new features are created here.
Feature engineering should be conducted on classification (file c).

"""
```

```
In [108... # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_1_explanations', value=feature_engineering_1_explanations)
```

feature_engineering_1_explanations

To show how the effect of feature engineering on baseline model , no new features are created here. Feature engineering should be conducted on classification (file c).

E. Data Preparation for Modeling

```
In [37]: # DO NOT MODIFY THE CODE IN THIS CELL
# Create copy of datasets
```

```
try:
    X_train = training_df_eng.copy()
    X_val = validation_df_eng.copy()
    X_test = testing_df_eng.copy()
except Exception as e:
    print(e)
```

E.1 Data Transformation

```
In [39]: # no data transformation is applied here.
```

```
In [109... data_transformation_1_explanations = """
Once this step is finished, the datasets are saved for baseline model and classification model (including feature engineering).
However, data after transformation generating multiple columns or scaled scores cause difficult for explanation when feature engineering,
As a result, this step is not a ideal place to apply feature engineering.
"""
```

```
In [110... # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_1_explanations', value=data_transformation_1_explanations)
```

data_transformation_1_explanations

Once this step is finished, the datasets are saved for baseline model and classification model (including feature engineering). However, data after transformation generating multiple columns or scaled scores cause difficult for explanation when feature engineering, As a result, this step is not a ideal place to apply feature engineering.

F. Save Datasets

Do not change this code

```
In [43]: # DO NOT MODIFY THE CODE IN THIS CELL

try:
    X_train.to_csv(at.folder_path / 'X_train.csv', index=False)

    X_val.to_csv(at.folder_path / 'X_val.csv', index=False)

    X_test.to_csv(at.folder_path / 'X_test.csv', index=False)
except Exception as e:
    print(e)
```